```
    rend = {round(P.t0 + P.h, 12): P for P in rp}; lows, ups, ts = [], [], []
    for P in pieces:
        te = round(P.t0 + P.h, 12); ts.append(te); ups.append(up[te])
        lows.append(max(l1_lower_nd(P.xs, end_values_nd(P), rend[te].xs, end_values_nd(rend[te])) - rb[te], 0.0) if te in rend and te in rb else 0.0)
    bad = [i for i, l in enumerate(lows) if l > eps]; good = all(e <= eps for e in ups); rep = {}
    nm = "xyz"[:d] if d <= 3 else [f"x{i}" for i in range(d)]
    if good: rep["판정"] = dict(status="증명", text=f"에러 없음: 모든 조각 끝에서 L¹ 오차 ≤ {eps:g}(증명, 가정 H 아래)")
    elif bad: rep["판정"] = dict(status="반례", text=f"정확도 에러: t = {ts[bad[0]]:.4g}에서 L¹ 오차 > {eps:g}(반례)")
    else:
        kk = int(np.argmax(np.array(ups) > eps))
        rep["판정"] = dict(status="증명", text=f"허용치 경계: t = {ts[kk]:.4g}에서 L¹ 오차가 [{lows[kk]:.3e}, {ups[kk]:.3e}]이고 ε = {eps:g}를 품는다(증명된 해상도 문장)")
    rep["누가"] = dict(status="정의", text=f"{name}(검은 상자: 가중치·손실·구조에 접근하지 않음). 상수로 명시: {d}차원 마디 {'×'.join(str(len(x)) for x in xs)}개 × 조각 {n}개 × 표본 시각 {K}개 = {ncon}개의 값; 표본점 일치 {delta:.1e}")
    rep["무엇을"] = dict(status="증명", text=f"t = {ts[-1]:.4g}에서 L¹ 오차 ∈ [{lows[-1]:.3e}, {ups[-1]:.3e}] (위: 잔차의 적분과 부등식, 아래: 인증된 기준해와의 차이)")
    k_ok = 0
    while k_ok < len(ups) and ups[k_ok] <= eps: k_ok += 1
    rep["언제"] = dict(status="증명" if not bad else "반례", text=(f"[0, {ts[k_ok - 1]:.4g}]에서 L¹ 오차 ≤ {eps:g}(증명)" if k_ok else f"첫 조각부터 ≤ {eps:g}가 증명되지 않는다")
                       + (f"; t = {ts[bad[0]]:.4g}에서 처음으로 > {eps:g}(반례)" if bad else ""))
    ck = dict(initial="초기 오차", time_scheme="시간 방향 결함", space="공간 방향 결함", residual="잔차", jump="조각 사이 도약", rounding="반올림")
    loc = lambda r: f"t ∈ [{r['t'][0]:.3g}, {r['t'][1]:.3g}], " + " × ".join(f"{nm[i]} ∈ [{a:.2g}, {b:.2g}]" for i, (a, b) in enumerate(r["x"])) + f", {ck.get(r['cause'], r['cause'])} (몫 {r['share']:.3f})"
    rep["어디서"] = dict(status="정의(합은 증명)", text="; ".join(loc(r) for r in rows[:2]))
    rep["왜"] = dict(status="정의(합은 증명)", text="적분 부등식의 원인별 몫: " + ", ".join(f"{ck.get(k, k)} {v:.3f}" for k, v in sorted(tot.items(), key=lambda kv: -kv[1]) if v > 5e-4))
    return rep, dict(good=good, bad=bad, rows=rows, tot=tot, ups=ups, lows=lows, ts=ts, res=res)


def certify_with_prescription_nd(u, spec, u0box, u0grad, xs, n, K, eps, ref, op=None, name="PINN", retrain=None, max_steps=5, nmax=None):
    """certify_with_prescription의 d차원 판(처방 규칙은 같다)."""
    rep, out = certify_nd(u, spec, u0box, u0grad, xs, n, K, eps, ref, op, name); how = []; uu, xx, nn = u, xs, n
    for step in range(max_steps):
        if out["good"]: break
        dom = max(out["tot"], key=out["tot"].get)
        if dom in ("initial", "space", "rounding", "jump") and (nmax is None or len(xx[0]) * 2 - 1 <= nmax):
            xx = [np.linspace(x[0], x[-1], 2 * len(x) - 1) for x in xx]; what = f"표본 마디를 축마다 {len(xx[0])}개로 늘림(PINN은 그대로)"
        elif dom == "time_scheme" and nn < 4 * n:
            nn = 2 * nn; what = f"시간 조각을 {nn}개로 늘림(PINN은 그대로)"
        elif retrain is not None:
            r0 = out["rows"][0]; uu = retrain((r0["t"][0], r0["t"][1], r0["x"])); nn = n
            what = "국소화가 짚은 구역 t ∈ [%.3g, %.3g], " % tuple(r0["t"]) + " × ".join("[%.2g, %.2g]" % tuple(b) for b in r0["x"]) + "에 콜로케이션 점을 더해 이어서 학습(대상 쪽)"
        else: break
        _, out = certify_nd(uu, spec, u0box, u0grad, xx, nn, K, eps, ref, op, name)
        how.append(f"{what} → " + ("모든 조각 끝에서 ≤ ε 증명" if out["good"] else ("반례 유지" if out["bad"] else "경계")))
    rep["어떻게"] = (dict(status="증명" if out["good"] else ("반례" if out["bad"] else "증명"), text="처방을 적용해 다시 판정: " + "; ".join(how) + ("(처방 확인)" if out["good"] else ""))
                     if how else dict(status="증명", text="처방 필요 없음(에러 없음이 증명됨)"))
    return rep, out
```

claim_tools.py — 남은 빈틈 셋의 보완: 공간 이산 가려내기, 차수 조건(부처 나무·달퀴스트·테일러 표), 입력 탐색과 축소(19절)

```
"""claim_tools: 상수로 잡는 에러 검사의 남은 빈틈 셋을 메운다(19절).

빈틈 1  주장이 없을 때 → 주장을 목록에서 가려낸다(identify_space).
        공간: 목록(FDM, FVM, FEM P1, 2차원 P1 삼각형)의 각 이산에 R1(4.5절)을 걸어 반례가 나지 않는 것만 남긴다.
        시간 쪽의 방법 목록 가려내기(1.7절)와 같은 논리이다.  남는 것이 없으면 "목록의 어느 이산과도 맞지 않는다"가 반례이다.
        목록의 상수는 공개 코드의 공식과 같다: FDM 가중치 = 포른베르그 알고리즘(findiff), P1 삼각형 = 코탄젠트 라플라시안
        (libigl cotmatrix), FVM = 두 점 플럭스.

빈틈 2  공식 자체가 틀렸을 때 → 공식을 차수 조건과 정확한 유리수로 대조한다.
        룽게–쿠타(부처 나무):  모든 뿌리 나무 t, |t| <= p 에 대해  b^T Phi(t) = 1 / gamma(t)     (nodepy
```

```
    의 order 검사와 같은 조건)
        선형 다단계(달퀴스트): C_0 = sum_j alpha_j = 0,
                               C_q = sum_j alpha_j j^q / q! - sum_j beta_j j^(q-1) / (q-1)! = 0  (q = 1..
    p)
        공간 스텐실(테일러 표): q계 도함수의 p차 근사 sum_j w_j u(x + o_j h) / h^q 이면
                               m_r = sum_j w_j o_j^r / r! = delta_{rq}  (r = 0..q+p-1)        (포른베르그
    , findiff)
        어긋난 조건 하나가 반례이고, 그 나무·차수·모멘트가 "어디서"이다.

빈틈 3  특정 조건에서만 드러나는 버그 → 입력 공간을 탐색하고 반례를 최소화한다(fuzz_find).
        속성 기반 시험(QuickCheck, Hypothesis)의 탐색과 축소를 쓰되, 판정(오라클)은 이 문서의 정확한 검
    사이다.
        그래서 찾은 입력 하나하나가 반례이고, 축소된 최소 입력이 "언제(어떤 조건에서)"이다."""
from fractions import Fraction as Fr
from math import factorial
import numpy as np

# ---------------------------------------------------------------- 빈틈 1: 공간 이산 가려내기
SPACE_LIBRARY_1D = ("FDM", "FVM", "FEM P1")


def identify_space(L, x):
    """목록의 각 공간 이산에 R1(관계식)을 건다.  반환 (남은 이산 목록, {이산: 첫 반례})."""
    import space_check as SC
    kinds = list(SPACE_LIBRARY_1D) + (["FEM P1 삼각형"] if isinstance(x, tuple) and len(x) == 2 else [])
    kept, refuted = [], {}
    for k in kinds:
        try:
            r = SC.check(L, x, k)
        except Exception as e:                      # 마디 배치가 그 이산의 전제(균일 등)와 안 맞으면
    그 자체가 배제 사유
            refuted[k] = dict(rule="전제", i=None, j=None, note=str(e)); continue
        rel = [v for v in r["viol"] if v["rule"] == "관계식"]
        if rel: refuted[k] = max(rel, key=lambda v: v["excess"])
        else: kept.append(k)
    return kept, refuted


# ---------------------------------------------------------------- 빈틈 2: 공식의 차수 조건
def trees(p):
    """차수 1..p의 뿌리 나무(자식 나무들의 정렬된 튜플)."""
    T = {1: [()]}
    for n in range(2, p + 1):
        out = set()
        def parts(rem, maxkey, acc):
            if rem == 0: out.add(tuple(sorted(acc))); return
            for m in range(1, rem + 1):
                for t in T[m]:
                    key = (m, t)
                    if maxkey is None or key <= maxkey: parts(rem - m, key, acc + [key])
        parts(n - 1, None, [])
        T[n] = sorted({tuple(t for _, t in sorted(c)) for c in out})
    return T


def _order(t): return 1 + sum(_order(c) for c in t)


def _gamma(t):
    g = _order(t)
    for c in t: g *= _gamma(c)
    return g


def _phi(t, A, s):
    """내부 가중치 벡터: 잎이면 1, 아니면 성분별로 prod_k (A phi(t_k))."""
    v = [Fr(1)] * s
    for c in t:
        w = _phi(c, A, s); Aw = [sum(A[i][j] * w[j] for j in range(s)) for i in range(s)]
        v = [v[i] * Aw[i] for i in range(s)]
    return v


def rk_order_check(A, b, p):
    """주장한 차수 p까지 부처 조건을 정확히 검사.  반환 (통과?, [(나무, 차수, b^T Phi, 1/gamma)] 위반 목
    록)."""
    A = [[Fr(a) for a in row] for row in A]; b = [Fr(x) for x in b]; s = len(b); bad = []
    for n, ts in trees(p).items():
        for t in ts:
            lhs = sum(b[i] * v for i, v in enumerate(_phi(t, A, s))); rhs = Fr(1, _gamma(t))
```

```
            if lhs != rhs: bad.append((t, n, lhs, rhs))
    return not bad, bad


def tree_str(t):
    return "τ" if t == () else "[" + ",".join(tree_str(c) for c in t) + "]"


def lmm_order_check(alpha, beta, p):
    """C_0..C_p = 0 (정확한 유리수).  반환 (통과?, [(q, C_q)] 위반)."""
    a = [Fr(x) for x in alpha]; b = [Fr(x) for x in beta]; bad = []
    for q in range(p + 1):
        if q == 0: C = sum(a)
        else: C = sum(a[j] * Fr(j) ** q / factorial(q) for j in range(len(a))) - sum(b[j] * Fr(j) ** (q - 1) / factorial(q - 1) for j in range(len(b)))
        if C != 0: bad.append((q, C))
    return not bad, bad


def stencil_check(offsets, weights, deriv, p):
    """q = deriv계 도함수의 p차 스텐실: m_r = sum_j w_j o_j^r / r! = delta_{rq}, r = 0..q+p-1."""
    bad = []
    for r in range(deriv + p):
        m = sum(Fr(w) * Fr(o) ** r for o, w in zip(offsets, weights)) / factorial(r)
        if m != (1 if r == deriv else 0): bad.append((r, m))
    return not bad, bad


def fornberg(offsets, deriv):
    """포른베르그 공식과 같은 결과(정확한 유리수): 모멘트 조건의 반데르몽드 계를 유리수로 푼다."""
    n = len(offsets); M = [[Fr(o) ** r / factorial(r) for o in offsets] for r in range(n)]; rhs = [Fr(1 if r == deriv else 0) for r in range(n)]
    for c in range(n):                                          # 가우스 소거(정확)
        piv = next(r for r in range(c, n) if M[r][c] != 0); M[c], M[piv] = M[piv], M[c]; rhs[c], rhs[piv] = rhs[piv], rhs[c]
        for r in range(n):
            if r != c and M[r][c] != 0:
                f = M[r][c] / M[c][c]; M[r] = [M[r][k] - f * M[c][k] for k in range(n)]; rhs[r] -= f * rhs[c]
    return [rhs[i] / M[i][i] for i in range(n)]


# ---------------------------------------------------------------- 빈틈 3: 조건부 버그 탐색과 최소화
def fuzz_find(strategy, fails, max_examples=400):
    """hypothesis.find: fails(입력)가 참인 입력을 찾고, 가장 단순한 것으로 줄인다.  못 찾으면 None
    (못 찾았다는 것은 '탐색한 입력에서는 반례가 없었다'일 뿐 증명이 아니다)."""
    from hypothesis import find, settings, HealthCheck
    from hypothesis.errors import NoSuchExample
    try:
        return find(strategy, fails, settings=settings(max_examples=max_examples, database=None, deadline=None, derandomize=True,
                                                       suppress_health_check=list(HealthCheck)))
    except NoSuchExample:
        return None


# ---------------------------------------------------------------- 목록을 넓히기(공개 목록)와 유한 범위 전수 확인
def identify_stencil(L, x, deriv=2, widths=(3, 5, 7, 9)):
    """균일 격자의 내부 행마다 스텐실을 포른베르그 가중치(정확한 유리수, findiff와 같은 공식)와 대조한다.
    반환: {행: '중심 (w)점 (차수)차'} 와 맞는 것이 없는 행 목록.  행의 가중치 = L_ij h^deriv, 오프셋 = j - i."""
    from space_check import exact, gamF, K_SP
    L = np.asarray(L, float); n = len(x); h = (exact(x[-1]) - exact(x[0])) / (n - 1); g = gamF(K_SP)
    ref = {w: fornberg(list(range(-(w // 2), w // 2 + 1)), deriv) for w in widths}
    found, unmatched = {}, []
    for i in range(n):
        nz = [j for j in range(n) if L[i, j] != 0]
        if not nz: continue                                                 # 빈 행(경계에서 제거된 자유도)은 대조하지 않는다
        lo, hi = min(nz) - i, max(nz) - i; w = max(-lo, hi) * 2 + 1
        if w not in ref or i - w // 2 < 0 or i + w // 2 >= n: continue      # 경계 근처 행은 대조하지 않는다(정의)
        wt = [exact(L[i, i + o]) * h ** deriv for o in range(-(w // 2), w // 2 + 1)]
        tol = g * sum(abs(v) for v in ref[w])
        if all(abs(a - b) <= tol for a, b in zip(wt, ref[w])): found[i] = f"중심 {w}점 {w - deriv + 1}차"
        else: unmatched.append(i)
    return found, unmatched
```

```
154
155
156 def exhaustive(inputs, fails):
157     """유한한 입력 범위를 전부 확인한다.  반환: 반례가 나는 입력 목록(비어 있으면 그 범위 전체에서 증명)
    ."""
158     return [a for a in inputs if fails(a)]
```

fig_mesh_graph.py — 그림 3과 4.4절의 확인 출력

```
  1 """그림 3과 4.4절의 확인 출력: 공간 이산과 시간 이산을 격자·망 그래프로 그리고, 수식 <-> 그림 왕복을 정
    확히 확인한다.
  2 수식은 각 패널 제목 아래에 그대로 적는다.  그래프의 뜻은 mesh_graph.py 머리말(식 (G))을 따른다."""
  3 import numpy as np
  4 import sympy as sp
  5 import matplotlib.pyplot as plt
  6 from viz import check_clip, BLUE, ORANGE, AQUA, GRAY, RED
  7 from mesh_graph import (R, Graph, fdm_1d, fvm_1d, fem_p1_1d, fdm_2d, fem_p1_tri, matrix_from_graph, grap
    h_from_matrix,
  8                         stencil_formula, structure, commute_check, spacetime_rothe, p1_interp)
  9 import operators as ops
 10
 11 out = []
 12 # ---------------------------------------------------------------- 1. 세 공간 이산: 그림 -> 수식 -> 그
    림 (정확한 왕복)
 13 Gs = [fdm_1d(6), fvm_1d(5), fem_p1_1d([0, R(1, 8), R(1, 4), R(1, 2), R(3, 4), 1])]
 14 ref = {"FDM": ops.fdm_vertex(6, 1.0)[0], "FVM": ops.fvm_cell(5, 1.0)[0], "FEM P1": ops.fem_p1(np.array([
    0, .125, .25, .5, .75, 1]))[0]}
 15 u = sp.symbols("u_0:6")
 16 for G in Gs:
 17     L = matrix_from_graph(G)                                # 그림 -> 수식 (식 (G))
 18     G2 = graph_from_matrix(L, G.V, G.pos)                   # 수식 -> 그림
 19     rt = (G2.T == G.T)
 20     diff = float(np.max(np.abs(np.array(L.tolist(), dtype=float) - ref[G.name])))
 21     m, r_, s_ = structure(G)
 22     out.append(f"[{G.name}] 그림->수식->그림 왕복 일치: {rt},  operators.py 행렬과의 차이: {diff:.1e}")
 23     out.append(f"    구조: 메츨러 {m}, 행합 0 {r_}, V·L 대칭 {s_}")
 24     out.append(f"    1행: (Lu)_1 = {stencil_formula(G, 1, u)}")
 25     out.append(f"    0행(경계): (Lu)_0 = {stencil_formula(G, 0, u)}")
 26 # ---------------------------------------------------------------- 2. 2차원: P1 직각 삼각형 망과 5점 F
    DM
 27 Gt, Gf = fem_p1_tri(4), fdm_2d(4)
 28 same_T = (Gt.T == Gf.T)
 29 dV = [(k, Gt.V[k], Gf.V[k]) for k in range(Gt.n) if Gt.V[k] != Gf.V[k]]
 30 out.append(f"[2차원] P1 삼각형 망과 5점 FDM의 변 무게 T 일치: {same_T} (대각선 무게는 정확히 0)")
 31 out.append(f"    마디 무게 V가 다른 마디는 모서리 {[k for k, _, _ in dV]}뿐:")
 32 out.append("    " + ", ".join(f"V_{k} = {a} 대 {b}" for k, a, b in dV))
 33 out.append(f"    내부 마디 5: (Lu)_5 = {stencil_formula(Gt, 5, sp.symbols('u_0:16'))}")
 34 # ---------------------------------------------------------------- 3. 그래프 꼴이 아닌 수식(반례): 풍
    상 이류 -(u_i - u_{i-1})/h
 35 try:
 36     h = R(1, 4); Lup = sp.Matrix(4, 4, lambda i, j: (-1 / h if i == j and i > 0 else 1 / h if j == i - 1
     else 0))
 37     graph_from_matrix(Lup, [h] * 4)
 38 except ValueError as e:
 39     out.append(f"[풍상 이류] 수식->그림 거부(반례): {e}")
 40 # ---------------------------------------------------------------- 4. 순서: 공간 먼저 = 시간 먼저(고정
     격자), 시간 먼저 + 움직이는 격자
 41 for G in Gs[:1] + Gs[2:]:
 42     out.append(f"[{G.name}] 고정 격자: 공간 먼저(선 방법+후방 오일러) = 시간 먼저(로테): {commute_check(
    G)}")
 43 grids = [fem_p1_1d([0, R(1, 4), R(1, 2), R(3, 4), 1]), fem_p1_1d([0, R(1, 8), R(1, 4), R(1, 2), 1]), fem
    _p1_1d([0, R(1, 16), R(1, 8), R(1, 4), R(1, 2), 1])]
 44 B, Gl = spacetime_rothe(lambda k: grids[k], R(1, 10), 2)
 45 P = p1_interp(grids[0].pos, grids[1].pos)
 46 out.append(f"[시간 먼저, 움직이는 격자] 시공간 행렬 {B.shape[0]}×{B.shape[1]}")
 47 out.append("    1층 마디 1(x = 1/8)로 오는 시간 변: " + ", ".join(f"0층 마디 {j}에서 {P[1, j]}" for j in
     range(P.shape[1]) if P[1, j] != 0))
 48 out.append("    보간 무게의 행합이 모두 1: " + str(all(sum(P[a, j] for j in range(P.shape[1])) == 1 for
    a in range(P.shape[0]))))
 49 print("\n".join(out))
 50 open("mesh_graph_out.txt", "w").write("\n".join(out) + "\n")
 51
 52 # ---------------------------------------------------------------- 그림 3
 53 fig, ax = plt.subplots(2, 2, figsize=(7.2, 5.0), gridspec_kw=dict(hspace=0.22, wspace=0.18, left=0.03, r
    ight=0.98, bottom=0.03, top=0.93))
 54 lab = lambda t: sp.latex(t).replace("\\frac", "\\dfrac") if False else str(t)
 55 # (가) 1차원 세 이산 = 같은 그래프 꼴
 56 a = ax[0, 0]
 57 for G, y, c in zip(Gs, (2.0, 1.0, 0.0), (BLUE, ORANGE, AQUA)):
```

```
    from mesh_graph import draw_graph
    draw_graph(a, G, c, y=y, fmt=lambda t: str(t), node_label=lambda k, G=G: str(G.V[k]))
    tt = a.text(-0.02, y + 0.3, f"{G.name}: " + (r"$T=1/h_e$" if G.name == "FEM P1" else r"$T=1/h$") + r", 마디 아래 $=V_i$", fontsize=6.6, color=c, fontweight="bold", va="bottom"); tt._ide_key = True
a.set_xlim(-0.05, 1.05); a.set_ylim(-0.75, 2.75); a.axis("off")
a.set_title("(가) 1차원 FDM·FVM·FEM = 가중 그래프 (V, T)", fontsize=8.2, fontweight="bold", loc="left")
a.text(0.5, -0.72, r"$(Lu)_i=\frac{1}{V_i}\sum_{j\sim i}T_{ij}\,(u_j-u_i)$", fontsize=7.6, ha="center", va="bottom")
# (나) 2차원 P1 삼각형 망: 코탄젠트 무게, 대각선 0
a = ax[0, 1]
for i in range(3):
    for j in range(3):
        a.plot([j, j + 1], [i, i + 1], color=GRAY, lw=0.7, ls=(0, (2, 2)), zorder=0)
draw_graph(a, Gt, BLUE, fmt=lambda t: str(t), ms=6)
a.text(1.62, 1.5 + 0.05, "0", fontsize=6, color=RED, ha="center", va="bottom")
a.set_xlim(-0.25, 3.45); a.set_ylim(-0.45, 3.2); a.set_aspect("equal"); a.axis("off")
a.set_title("(나) 2차원 P1 삼각형 망 = 5점 FDM 그래프", fontsize=8.2, fontweight="bold", loc="left")
a.text(1.5, -0.42, r"$T_{ij}=\frac{\cot\alpha_{ij}+\cot\beta_{ij}}{2}$, 점선(대각선) $T=0$", fontsize=7.0, ha="center", va="bottom")
# (다) 공간 먼저: 고정 격자, 시간 변 = 1/tau
a = ax[1, 0]; Gf1 = fem_p1_1d([0, R(1, 4), R(1, 2), R(3, 4), 1])
for n in range(3):
    draw_graph(a, Gf1, AQUA, y=0.45 * n, label_edges=False, ms=5)
    a.text(-0.06, 0.45 * n, f"n = {n}", fontsize=6.2, ha="right", va="center")
    if n < 2:
        for p in Gf1.pos: a.annotate("", xy=(float(p), 0.45 * (n + 1) - 0.05), xytext=(float(p), 0.45 * n + 0.05), arrowprops=dict(arrowstyle="->", color=ORANGE, lw=0.8))
a.set_xlim(-0.2, 1.05); a.set_ylim(-0.32, 1.12); a.axis("off")
a.set_title("(다) 공간 먼저(선 방법) + 후방 오일러", fontsize=8.2, fontweight="bold", loc="left")
a.text(0.42, -0.30, r"$\frac{u_i^{n+1}-u_i^{n}}{\tau}=(L\,u^{n+1})_i$, 시간 변 무게 $1/\tau$", fontsize=7.0, ha="center", va="bottom")
# (라) 시간 먼저: 걸음마다 격자가 바뀜, 시간 변 = P1 보간 무게
a = ax[1, 1]
for n, G in enumerate(grids):
    draw_graph(a, G, AQUA, y=0.45 * n, label_edges=False, ms=5)
    a.text(-0.06, 0.45 * n, f"n = {n}", fontsize=6.2, ha="right", va="center")
for n in range(2):
    Pn = p1_interp(grids[n].pos, grids[n + 1].pos)
    for aa in range(Pn.shape[0]):
        for j in range(Pn.shape[1]):
            if Pn[aa, j] != 0:
                a.annotate("", xy=(float(grids[n + 1].pos[aa]), 0.45 * (n + 1) - 0.05), xytext=(float(grids[n].pos[j]), 0.45 * n + 0.05),
                           arrowprops=dict(arrowstyle="->", color=ORANGE, lw=0.4 + 0.8 * float(Pn[aa, j])))
a.set_xlim(-0.2, 1.05); a.set_ylim(-0.32, 1.12); a.axis("off")
a.set_title("(라) 시간 먼저(로테), 걸음마다 새 격자", fontsize=8.2, fontweight="bold", loc="left")
a.text(0.42, -0.30, r"$\frac{u^{n+1}-P_n u^{n}}{\tau}=L_{n+1}u^{n+1}$, 시간 변 무게 = 보간 $P_n$", fontsize=7.0, ha="center", va="bottom")
for a_ in ax.flat: a_.set_xticks([]); a_.set_yticks([])
check_clip(fig, "fig_mesh_graph")
fig.savefig("fig_mesh_graph.png", dpi=220)
```

nonlinear.py — 결함 적분 한도와 이산 비교 원리

```
"""nonlinear: a posteriori bounds that need no linearization.
  scalar_chain(traj, h, F, Fabs, ell, wts)   one-sided Lipschitz constant ell (proved algebraically by the user):
        e_{k+1} <= e^{ell h} e_k + sqrt((e^{2 ell h} - 1)/(2 ell)) (int_0^h ||d||^2)^{1/2} + jump_k,  d = p' - F(p),
        p the cubic Hermite interpolant of any scheme's outputs (Gauss-Legendre is exact for the polynomial ||d||^2)
  comparison(traj, h, F, defect_poly, majorant, n, delta, xw, alpha, c, wnorm)   cooperative systems (Metzler Jacobian):
        |e| <= E componentwise, E' <= M E + g with a Metzler majorant M (bootstrap level delta) and the componentwise
        Bernstein bound g of the defect; non-negative propagators keep the order (prop_upper: upper bound of e^{hA} z)
Rounding: standard model, gam_k = k u / (1 - k u), added to every evaluated quantity."""
import json, math
import numpy as np
from math import comb
from numpy.polynomial.legendre import leggauss

U = 2.0 ** -53
gam = lambda k: k * U / (1 - k * U)


def scalar_chain(traj, h, F, Fabs, ell, wts, nodes=12):
```

```
    xg, wg = leggauss(nodes); sn = 0.5 * h * (xg + 1); wn = 0.5 * h * wg
    nw = lambda z: float(np.sqrt(np.sum(wts * z * z)))
    grow = math.exp(ell * h) * (1 + 4 * U)
    wfac = (math.sqrt((math.exp(2 * ell * h) - 1) / (2 * ell)) if ell > 0 else math.sqrt(h)) * (1 + 4 *
U)
    e = 0.0
    for a, b in zip(traj[:-1], traj[1:]):
        fa, fb = F(a), F(b); dl = b - a
        c2 = 3 * dl / h ** 2 - (2 * fa + fb) / h; c3 = -2 * dl / h ** 3 + (fa + fb) / h ** 2
        I2 = 0.0
        for s, w in zip(sn, wn):
            p = a + s * (fa + s * (c2 + s * c3)); dp = fa + s * (2 * c2 + 3 * s * c3); I2 += w * nw(dp -
 F(p)) ** 2
        cs = [a, fa, c2, c3]
        Pb = sum(np.abs(cj) * h ** j for j, cj in enumerate(cs)); dPb = sum(j * np.abs(cj) * h ** (j - 1
) for j, cj in enumerate(cs) if j)
        D = math.sqrt(I2 * (1 + gam(40))) + gam(40) * (nw(dPb) + nw(Fabs(Pb))) * math.sqrt(h)
        J = nw(a + h * (fa + h * (c2 + h * c3)) - b) + gam(8) * nw(Pb)
        e = grow * e + wfac * D + J
    return e


def prop_upper(A, z, h, n=8):
    """componentwise upper bound of e^{hA} z for Metzler A and z >= 0"""
    th = h * float(np.max(np.sum(np.abs(A), axis=1))); m = max(1, math.ceil(th / 0.5)); hs = h / m
    absA = np.abs(A); nnz = int(np.max(np.sum(A != 0, axis=1))); K = n * (nnz + 2) + n
    for _ in range(m):
        c = z.copy(); S = z.copy(); a = z.copy(); asum = z.copy()
        for j in range(1, n + 1):
            c = (A @ c) * hs / j; S = S + c
            a = (absA @ a) * hs / j; asum = asum + a
        tail = (absA @ a) * hs / (n + 1) / (1 - (th / m) / (n + 2))
        z = np.maximum(S, 0.0) + tail + gam(K) * asum
    return z


def pmul(P, Q):
    out = np.zeros((P.shape[0] + Q.shape[0] - 1, P.shape[1]))
    for i in range(P.shape[0]):
        for j in range(Q.shape[0]):
            out[i + j] += P[i] * Q[j]
    return out


pder = lambda P: np.array([j * P[j] for j in range(1, P.shape[0])])


def bern(P, h):
    m = P.shape[0] - 1; a = np.array([P[j] * h ** j for j in range(m + 1)])
    return np.array([sum(comb(k, j) / comb(m, j) * a[j] for j in range(k + 1)) for k in range(m + 1)])


def hermite(a, b, fa, fb, h):
    dl = b - a
    return np.array([a, fa, 3 * dl / h ** 2 - (2 * fa + fb) / h, -2 * dl / h ** 3 + (fa + fb) / h ** 2])


mag = lambda P, h: np.sum(np.abs(P) * (h ** np.arange(P.shape[0]))[:, None], axis=0)


def rk4_traj(F, y, h, N):
    out = [y.copy()]
    for _ in range(N):
        k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + h / 2 * k2); k4 = F(y + h * k3)
        y = y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4); out.append(y.copy())
    return out


def comparison(traj, h, F, defect_poly, majorant, n, delta, xw, alpha, c, wnorm):
    """xw: position of every state component (for the weight); alpha*c > 0 or alpha = 0"""
    Dw = np.exp(alpha * xw); Et = np.zeros(n); maxE = 0.0; t = 0.0
    for a, b in zip(traj[:-1], traj[1:]):
        Pc = hermite(a, b, F(a), F(b), h)
        dP, magd = defect_poly(Pc)
        g = np.max(np.abs(bern(dP, h)), axis=0) + gam(80) * mag(magd, h)
        M = majorant(Pc, delta)
        Mt = (Dw[:, None] * M) / Dw[None, :] - alpha * c * np.eye(n)
        gt = Dw * math.exp(-alpha * c * t) * g
        sig = max(float(gt.max()), 1e-300)
        Ma = np.zeros((n + 1, n + 1)); Ma[:n, :n] = Mt; Ma[:n, n] = gt / sig
```

```
        z = prop_upper(Ma, np.append(Et, sig), h)
        pend = Pc[0] + h * (Pc[1] + h * (Pc[2] + h * Pc[3]))
        J = np.abs(pend - b) + gam(8) * mag(Pc, h)
        t += h
        wt = Dw * math.exp(-alpha * c * t)
        Et = z[:n] + wt * J
        maxE = max(maxE, float(np.max(Et / wt)))
    E = Et / (Dw * math.exp(-alpha * c * t))
    return E, maxE, wnorm(E)
```

pde_judge.py — 경로 B: PDE 수준 L^1 사후 판정과 국소화(시간 스킴 몫·공간 몫)

```
"""pde_judge: a posteriori L1 bound against the CONTINUOUS solution of a reaction-diffusion system on [0, Lx],
    u_t = D_a u_a,xx + R_a(u),  a = 1..m,  R polynomial,
with zero-flux or Robin ends (-D u_x = kap (c - u) at x = 0) for the diffusing species.

The candidate is any piecewise polynomial in (x, t): a sequence of pieces, each a time step [t0, t0 + h] on its own node
set x with nodal time coefficients V[a] (N, K) in the local time s.  It may come from the method of lines (space first),
Rothe's method (time first, moving meshes) or a space-time Galerkin method; the judge does not ask.

Reconstruction: in x the nodal values are joined by the clamped C2 cubic spline whose end slopes satisfy the boundary
conditions (zero flux: slope 0 exactly; Robin: slope -(kap/D)(c - value), exact up to a rounding term beta that is bounded).
Residual: r_a = u~_a,t - D_a u~_a,xx - R_a(u~), a polynomial on every element x step rectangle; its L1 norm is bounded by
the mean absolute 2D Bernstein coefficient times the area, plus the rounding of forming it.

Theorem (L1 bound).  Assume (H) the exact solution stays in a box B = [lo, hi] (an invariant region, supplied by the
user with its proof).  Let mu_k be an upper bound, over the convex hull of B and the candidate's range on piece k, of the
L1 logarithmic norm of the Jacobian, mu_1(J) = max_b (J_bb + sum_{a != b} |J_ab|) (interval evaluation).  With e = u - u~,
R(u) - R(u~) = A e, A = int_0^1 J(u~ + th e) d th (mean value theorem); the test function sgn(e_a) in the weak form gives
Kato's inequality for the diffusion part (boundary terms <= |beta|) and sum_a sgn(e_a)(A e)_a <= mu_k sum_b |e_b|, so
    d/dt ||e||_1 <= mu_k ||e||_1 + ||r||_1 + |beta|   on piece k,
and with E_0 = ||e(0)||_1,  E_{k+1} = e^{max(mu_k, 0) h_k} (E_k + iint_k |r| + int_k |beta|) + ||u~(t_{k+1}+) - u~(t_{k+1}-)||_1.
When J has non-negative off-diagonal entries and zero column sums (an exchange structure) mu_k = 0 up to rounding.

Localization (localize): unrolling the recursion, bound = W_0 e0 + sum_k W_k (jump_k + sum_{a,cell} rho_{k,a,cell} + beta_k),
W_k = prod_{i >= k} g_i, every term non-negative: the bound splits exactly into (piece, cell, species, cause) shares.  With op,
each cell residual is divided between the time scheme (spline of the nodal semi-discrete defect) and space (the rest)."""
import json, math, time
import numpy as np
import sympy as sp
from math import comb
from scipy.linalg import solve_banded
from core import I
from rig import gam

Hb = np.array([[1, 0, -3, 2], [0, 1, -2, 1], [0, 0, 3, -2], [0, 0, -1, 1]], float)   # H00, H10, H01, H11 in xi powers


def bern_matrix(m):
    M = np.zeros((m + 1, m + 1))
    for k in range(m + 1):
        for j in range(k + 1): M[k, j] = comb(k, j) / comb(m, j)
    return M


B3 = bern_matrix(3)


class Q:
    """polynomials in (xi, s) on a batch of rectangles: coefficient array (cells, px, ps)"""
    def __init__(s, c): s.c = np.asarray(c, float)
    @staticmethod
```

```
    def k(o, like):
        if isinstance(o, Q): return o
        c = np.zeros((like.c.shape[0], 1, 1)); c[:, 0, 0] = o; return Q(c)
    def _pad(s, sh):
        Z = np.zeros((s.c.shape[0],) + sh); Z[:, :s.c.shape[1], :s.c.shape[2]] = s.c; return Z
    def __add__(s, o):
        o = Q.k(o, s); sh = (max(s.c.shape[1], o.c.shape[1]), max(s.c.shape[2], o.c.shape[2])); return Q(s._pad(sh) + o._pad(sh))
    __radd__ = __add__
    def __neg__(s): return Q(-s.c)
    def __sub__(s, o): return s + (-Q.k(o, s))
    def __rsub__(s, o): return Q.k(o, s) + (-s)
    def __mul__(s, o):
        if not isinstance(o, Q): return Q(s.c * float(o))
        n, p, q = s.c.shape; _, r, t = o.c.shape; C = np.zeros((n, p + r - 1, q + t - 1))
        for i in range(p):
            for j in range(r):
                for k in range(q): C[:, i + j, k:k + t] += s.c[:, i, k][:, None] * o.c[:, j, :]
        return Q(C)
    __rmul__ = __mul__
    def __pow__(s, n):
        out = Q(np.ones((s.c.shape[0], 1, 1)))
        for _ in range(int(n)): out = out * s
        return out


class Piece:
    def __init__(s, t0, h, x, V):
        s.t0, s.h, s.x, s.V = t0, h, np.asarray(x, float), [np.asarray(v, float) for v in V]   # V[a]: (N, K)


def spline_slopes(x, y, s0, s1):
    e = np.diff(x); n = len(x) - 2; ab = np.zeros((3, n))
    ab[1] = 2 * (e[:-1] + e[1:]); ab[2, :-1] = e[2:]; ab[0, 1:] = e[:-2]
    rhs = 3 * (e[1:][:, None] * (y[1:-1] - y[:-2]) / e[:-1][:, None] + e[:-1][:, None] * (y[2:] - y[1:-1]) / e[1:][:, None])
    rhs[0] -= e[1] * s0; rhs[-1] -= e[-2] * s1
    return np.vstack([s0[None], solve_banded((1, 1), ab, rhs), s1[None]])


def cubic_coeffs(x, y, sl):
    ell = np.diff(x); a, b, sa, sb = y[:-1], y[1:], sl[:-1] * ell[:, None], sl[1:] * ell[:, None]
    return (Hb[0][None, :, None] * a[:, None, :] + Hb[1][None, :, None] * sa[:, None, :]
            + Hb[2][None, :, None] * b[:, None, :] + Hb[3][None, :, None] * sb[:, None, :])


def _exact_inverse(nodes):
    from fractions import Fraction as F_
    Vq = sp.Matrix([[sp.Rational(F_(float(e)).numerator, F_(float(e)).denominator) ** k for k in range(4)] for e in nodes])
    return np.array(Vq.inv().evalf(30).tolist(), dtype=float)


ETA = np.array([0.0, 1 / 3, 2 / 3, 1.0]); VI = _exact_inverse(ETA); VINORM = float(np.abs(VI).sum(1).max()); VIERR = 2.0 ** -53 * VINORM


def l1_diff(xA, cA, xB, cB):
    """upper bound of int |pA - pB| for two piecewise cubics (xi-power coefficients (cells, 4)) on different meshes:
    union-of-knots cells, values at 4 stored nodes, exact-rational inverse Vandermonde (rounded), Bernstein mean"""
    xu = np.union1d(xA, xB); xu = xu[(xu >= max(xA[0], xB[0])) & (xu <= min(xA[-1], xB[-1]))]; a, b = xu[:-1], xu[1:]
    vals = []; cs = []
    for X, C in ((xA, cA), (xB, cB)):
        j = np.clip(np.searchsorted(X, 0.5 * (a + b)) - 1, 0, len(X) - 2); ell = X[j + 1] - X[j]
        xi = (a[:, None] + (b - a)[:, None] * ETA[None, :] - X[j][:, None]) / ell[:, None]
        vals.append(np.einsum("nk,nmk->nm", C[j], xi[:, :, None] ** np.arange(4)[None, None, :])); cs.append(np.abs(C[j]).sum(1))
    D = (vals[0] - vals[1]) @ VI.T; bz = D @ B3.T
    ev = gam(16) * (cs[0] + cs[1]) + gam(4) * (np.abs(vals[0]) + np.abs(vals[1])).max(1)
    tol = (VINORM * ev + (VIERR + gam(8) * VINORM) * (np.abs(vals[0]).max(1) + np.abs(vals[1]).max(1) + ev)) * 4 + gam(8) * np.abs(D).sum(1) * 2
    return float(np.sum((b - a) * (np.mean(np.abs(bz), axis=1) + tol)))


class Judge:
    def __init__(s, spec):
```

```
        s.spec = spec; s.m = m = spec["m"]; s.D = list(spec["D"]); s.T = spec["T"]
        syms = sp.symbols(f"s0:{m}"); Rs = [sp.expand(sp.sympify(e)) for e in spec["R"](list(syms))]
        s.R = sp.lambdify(syms, Rs, modules="math")
        Rabs = []
        for e_ in Rs:
            pe = sp.Poly(e_, *syms) if e_.free_symbols else None
            Rabs.append(sum(abs(c) * sp.prod([sy ** k for sy, k in zip(syms, mon)]) for mon, c in pe.ter
ms()) if pe else abs(e_))
        s.Rabs = sp.lambdify(syms, Rabs, modules="math")
        J = [[sp.diff(Rs[a], syms[b]) for b in range(m)] for a in range(m)]
        s.J = [[sp.lambdify(syms, J[a][b], modules="math") for b in range(m)] for a in range(m)]
        # column measures J_bb + sum_{a != b} sign_ab J_ab for every sign pattern (sign certified on a s
ub-box), simplified
        # symbolically so that the interval evaluation keeps the cancellations (dependency problem)
        import itertools
        s.colexpr = {}
        for b in range(m):
            others = [a for a in range(m) if a != b]
            for pat in itertools.product((1, -1), repeat=len(others)):
                e_ = J[b][b] + sum(sg * J[a][b] for a, sg in zip(others, pat) if sg != 0)
                s.colexpr[(b, pat)] = sp.lambdify(syms, sp.simplify(e_), modules="math")
        s.bc = spec.get("bc", [dict(left=("neumann",))] * m)
        s.lo, s.hi = np.asarray(spec["box"][0], float), np.asarray(spec["box"][1], float)

    def _slopes(s, a, x, Va):
        K = Va.shape[1]; e0v = np.eye(1, K, 0)[0]; L = s.bc[a].get("left", ("neumann",))
        if s.D[a] > 0 and L[0] == "robin":
            kap, c = L[1], L[2]; s0 = -(kap / s.D[a]) * (c * e0v - Va[0])
            beta = gam(8) * (s.D[a] * np.abs(s0) + kap * (c * e0v + np.abs(Va[0])))
        else:
            s0 = np.zeros(K); beta = np.zeros(K)
        return spline_slopes(x, Va, s0, np.zeros(K)), beta

    def mu1(s, lo, hi, G=16):
        """upper bound of max over the box of mu_1(J) = max_b (J_bb + sum_{a!=b} |J_ab|): the box is cut
 into G^m sub-boxes;
        on each, an off-diagonal entry with a certified sign enters the symbolic column sum with that si
gn, and an uncertain one
        through |x| = max(x, -x) (both symbolic sums are evaluated), so cancellations are not lost to in
terval dependency"""
        m = s.m; G = G if m <= 3 else 4
        grids = [np.linspace(lo[a], hi[a], G + 1) for a in range(m)]
        idx = np.array(np.meshgrid(*[np.arange(G)] * m, indexing="ij")).reshape(m, -1)
        box = [I(grids[a][idx[a]], grids[a][idx[a] + 1]) for a in range(m)]
        nb_ = idx.shape[1]; ones = np.ones(nb_)
        val = lambda v: v if isinstance(v, I) else I(np.asarray(v, float) * ones)
        Jv = [[val(s.J[a][b](*box)) for b in range(m)] for a in range(m)]
        best = -np.inf
        for b in range(m):
            others = [a for a in range(m) if a != b]
            sign = [np.where(Jv[a][b].lo >= 0, 1, np.where(Jv[a][b].hi <= 0, -1, 0)) for a in others]
            pats = np.stack(sign, 1) if others else np.zeros((nb_, 0), int)
            import itertools
            for pat in {tuple(p_) for p_ in pats}:
                sel = np.all(pats == np.array(pat), axis=1) if others else np.ones(nb_, bool)
                sub = [I(bx.lo[sel], bx.hi[sel]) for bx in box]; ns = int(sel.sum())
                # |J_ab| = max(J_ab, -J_ab): an uncertain sign is resolved by taking the larger of both
symbolic sums
                choices = [(sg,) if sg != 0 else (1, -1) for sg in pat]
                for full in itertools.product(*choices) if others else [()]:
                    v = s.colexpr[(b, tuple(full))](*sub)
                    hi_ = (v.hi * np.ones(ns)) if isinstance(v, I) else np.full(ns, float(v) * (1 + 1e-1
5) + 1e-300)
                    best = max(best, float(np.max(hi_)))
        return best

    def run(s, pieces, u0box, nsub=256, op=None, series_every=25):
        """u0box(a, xlo, xhi) -> (lo, hi): rigorous enclosure of the exact initial data of species a on
[xlo, xhi].
        op(x) -> (L, w, src) (optional): the spatial operator on the node set x.  When given, every cell
 residual is also
        split into its time-scheme part and its spatial part (see localize); the bound itself does not u
se op."""
        from core import P as _P
        m = s.m; Eb = 0.0; rnd = 0.0; bsum = 0.0; jumps = 0.0; resid = 0.0; e0 = None; prev = None; seri
es = []; mu_max = -np.inf; t0w = time.time()
        Mc = {}; k_ = 0; LOC = []
        for Pc in pieces:
            x, h = Pc.x, Pc.h; K = Pc.V[0].shape[1]; ell = np.diff(x); tlen = np.array([h ** j for j in
range(4 * K)])
```

```
            Q_ = []; beta_tot = 0.0; tot = 0.0; cell = []; dom = []; rnd_p = 0.0
            for a in range(m):
                sl, beta = s._slopes(a, x, Pc.V[a]); Q_.append(Q(cubic_coeffs(x, Pc.V[a], sl))); beta_to
t += h * float(beta @ tlen[:K])
            if e0 is None:                                      # initial error of the first piece
                e0 = 0.0; d = 1.0 / nsub; a0 = np.arange(nsub) / nsub
                for a in range(m):
                    cu = Q_[a].c[:, :, 0]
                    for j in range(len(x) - 1):
                        c = cu[j]; xs = x[j] + ell[j] * np.arange(nsub + 1) / nsub
                        C4 = np.stack([c[0] + c[1] * a0 + c[2] * a0 ** 2 + c[3] * a0 ** 3, (c[1] + 2 * c
[2] * a0 + 3 * c[3] * a0 ** 2) * d,
                                       (c[2] + 3 * c[3] * a0) * d * d, np.full(nsub, c[3] * d ** 3)], 1)
                        bz = C4 @ B3.T; tl = 1e-13 * np.abs(C4).sum(1); clo, chi = bz.min(1) - tl, bz.ma
x(1) + tl
                        ulo, uhi = uObox(a, xs[:-1], xs[1:])
                        e0 += float(np.sum(np.maximum(np.abs(uhi - clo), np.abs(chi - ulo)) * ell[j] * d
))
                Eb = e0
            dt = lambda A: Q(np.concatenate([A.c[:, :, 1:] * np.arange(1, A.c.shape[2])[None, None, :],
np.zeros(A.c.shape[:2] + (1,))], 2))
            dxx = lambda A: Q(np.concatenate([A.c[:, 2:, :] * (np.arange(2, A.c.shape[1]) * np.arange(1,
 A.c.shape[1] - 1))[None, :, None],
                                              np.zeros((A.c.shape[0], 2, A.c.shape[2]))], 1) / (ell ** 2
)[:, None, None])
            Rv = s.R(*Q_); Ra = s.Rabs(*[Q(np.abs(q.c)) for q in Q_])
            for a in range(m):
                ra = dt(Q_[a]) - s.D[a] * dxx(Q_[a]) - Rv[a]
                mag = Q(np.abs(dt(Q_[a]).c)) + s.D[a] * Q(np.abs(dxx(Q_[a]).c)) + Ra[a]
                px, ps = ra.c.shape[1:]
                if (px, ps) not in Mc: Mc[(px, ps)] = (bern_matrix(px - 1), bern_matrix(ps - 1))
                Mx, Ms = Mc[(px, ps)]; Rs_ = ra.c * tlen[None, None, :ps]
                Bc = np.einsum("ij,njk,lk->nil", Mx, Rs_, Ms)
                cv = ell * h * (np.mean(np.abs(Bc), axis=(1, 2)) + 1e-13 * np.sum(np.abs(Rs_), axis=(1,
2)))
                tot += float(np.sum(cv)); cell.append(cv)
                if op is not None:
                    # time-scheme part r_t: the nodal defect of the semi-discrete system on this mesh, d
elta_a = p_a' - D_a L p_a
                    # - src_a - R_a(p) (a polynomial in s at every node), joined in x by the same spline
; spatial part r_x = r - r_t
                    if a == 0:
                        Lop, _, src = op(x); Pn = [_P(Pc.V[b].T) for b in range(m)]; Rn = [_P.k(v, len(x
)) for v in s.R(*Pn)]
                    dl = Pn[a].der() - _P((Lop @ Pc.V[a]).T * s.D[a]) - _P.k(np.asarray(src, float).resh
ape(m, -1)[a], len(x)) - Rn[a]
                    rt = Q(cubic_coeffs(x, dl.c.T, spline_slopes(x, dl.c.T, np.zeros(dl.c.shape[0]), np.
zeros(dl.c.shape[0]))))
                    def l1c(A):
                        Rs2 = A.c * tlen[None, None, :A.c.shape[2]]
                        return ell * h * np.mean(np.abs(np.einsum("ij,njk,lk->nil", bern_matrix(A.c.shap
e[1] - 1), Rs2, bern_matrix(A.c.shape[2] - 1))), axis=(1, 2))
                    at, ax = l1c(rt), l1c(ra - rt)
                    dom.append(np.where(at + ax > 0, at / np.maximum(at + ax, 1e-300), 0.5))
                else:
                    dom.append(None)
                rr = gam(400) * 2 * float(np.sum(ell * h * np.sum(mag.c[:, :px, :ps] * tlen[None, None,
:ps], axis=(1, 2))))
                tot += rr; rnd += rr; rnd_p += rr
            resid += tot - 0.0; bsum += beta_tot
            # L1 log-norm of the Jacobian on the hull of the invariant box and the candidate's range on
this piece
            lo, hi = s.lo.copy(), s.hi.copy()
            for a in range(m):
                Us = Q_[a].c * tlen[None, None, :K]; bz = np.einsum("ij,njk,lk->nil", B3, Us, bern_matri
x(K - 1)); tl = 1e-13 * np.abs(Us).sum()
                lo[a] = min(lo[a], bz.min() - tl); hi[a] = max(hi[a], bz.max() + tl)
            mu = s.mu1(lo, hi)
            mu_max = max(mu_max, mu)
            pending = (mu, tot, beta_tot)
            # jump from the previous piece
            starts = [q.c[:, :, 0] for q in Q_]
            if prev is not None:
                xp, ends, rads = prev
                jl = [l1_diff(xp, ends[a], x, starts[a]) + rads[a] for a in range(m)]; jmp = sum(jl); ju
mps += jmp
                Eb_prev_jump = jmp
            else: Eb_prev_jump = 0.0; jl = [0.0] * m
            mu, tot_, bt_ = pending
            g = math.exp(max(mu, 0.0) * h) * (1 + 1e-15)
```

```
            LOC.append(dict(t0=Pc.t0, h=h, x=x, cell=cell, dom=dom, jump=jl, rounding=rnd_p + bt_, g=g))
            Eb = g * (Eb + Eb_prev_jump + tot_ + bt_)
            ends = [np.einsum("nmk,k->nm", q.c, tlen[:K]) for q in Q_]
            rads = [gam(2 * K + 4) * float(np.sum(ell * np.sum(np.abs(q.c) * tlen[None, None, :K], axis=(1, 2)))) for q in Q_]
            prev = (x, ends, rads); k_ += 1; tnow = Pc.t0 + h
            if k_ % series_every == 0: series.append([tnow, Eb])
        series.append([tnow, Eb])
        W = 1.0                                                 # weight of piece k in the final bound: prod_{j >= k} g_j
        for L_ in reversed(LOC): W *= L_["g"]; L_["W"] = W
        return dict(loc=LOC, pieces=k_, e0=e0, residual=resid - rnd, rounding=rnd + bsum, jumps=jumps, bound=Eb * (1 + 1e-10), mu1_max=mu_max,
                    series=series, wall=time.time() - t0w)


def localize(res, top=10):
    """where and how, exactly additive: final bound / (1 + 1e-10) = W_1 e0 + sum_k W_k (jump_k + residual_k + rounding_k),
    residual_k = sum over species and cells of the cell L1 residual.  Each entry is (time piece, x cell, species, cause) with
    its share of the bound.  When run() was given op, a cell's residual share is divided between "time scheme" and "space"
    in proportion to the L1 norms of r_t and r_x on that cell (an allocation: the two parts still add up to the cell share).
    Returns the rows sorted by share and the per-cause totals (they add up to 1)."""
    B = res["bound"] / (1 + 1e-10); L = res["loc"]; rows = []
    tot = dict(initial=L[0]["W"] * res["e0"], time_scheme=0.0, space=0.0, residual=0.0, jump=0.0, rounding=0.0)
    for P_ in L:
        W = P_["W"]; x = P_["x"]
        for a, (cv, ft) in enumerate(zip(P_["cell"], P_["dom"])):
            parts = [("residual", cv)] if ft is None else [("time_scheme", cv * ft), ("space", cv * (1 - ft))]
            for name, arr in parts:
                for j in np.argsort(-arr)[:top]:
                    rows.append(dict(t=[P_["t0"], P_["t0"] + P_["h"]], x=[float(x[j]), float(x[j + 1])], species=a, cause=name,
                                     share=W * float(arr[j]) / B))
                tot[name] += W * float(np.sum(arr))
        for a, jv in enumerate(P_["jump"]):
            if jv > 0: rows.append(dict(t=[P_["t0"], P_["t0"]], x=None, species=a, cause="jump", share=W * jv / B))
            tot["jump"] += W * jv
        tot["rounding"] += W * P_["rounding"]
    rows.sort(key=lambda r: -r["share"])
    return rows[:top], {k: v / B for k, v in tot.items() if v > 0 or k in ("initial", "jump")}
```

orders.py — 시간 먼저(로테)와 시공간 동시(dG) 후보

```
"""orders: candidates produced in the other two orders of discretization, as pde_judge pieces.
  rothe(prob, N, ...)   TIME FIRST (Rothe's method): every step of the 2-stage Radau IIA scheme (order 3, L-stable) is an
                        elliptic problem in x, solved by the spatial operator of that step's mesh; the mesh is re-equidistributed
                        (arc-length monitor of the current solution) every `remesh` steps and the solution is carried over by
                        nodal interpolation of the spline.  The candidate on a step is the collocation polynomial.
  stdg(prob, N, q, n)   SPACE AND TIME TOGETHER: space-time Galerkin on time slabs, the spatial operator in x times
                        discontinuous polynomials of degree q in t (dG(q)), test functions tau^i, time integrals by
                        Gauss-Legendre with q+3 points (exact for quadratic reaction terms).
prob = dict(m, D, R, T, u0 (list of functions of x), op (x -> (L, w, src)), robin (None or (kap, c) for species 0 at x=0)).
Both return pieces for pde_judge.Judge.run, plus the mesh history."""
import math
import numpy as np
import sympy as sp
from scipy.linalg import solve
from pde_judge import Piece, spline_slopes, cubic_coeffs


class RD:
    """right-hand side F(z) = D_a L u_a + R_a(u) + src and its Jacobian, for a given spatial operator"""
    def __init__(s, prob):
        s.m = prob["m"]; s.D = prob["D"]; syms = sp.symbols(f"s0:{s.m}"); Rs = [sp.sympify(e) for e in prob["R"](list(syms))]
```

```
        s.Rn = sp.lambdify(syms, Rs, "numpy"); s.Jn = [[sp.lambdify(syms, sp.diff(Rs[a], syms[b]), "nump
y") for b in range(s.m)] for a in range(s.m)]

    def FJ(s, z, L, src):
        m, N = s.m, L.shape[0]; u = z.reshape(m, N); r = s.Rn(*u)
        F = np.concatenate([s.D[a] * (L @ u[a]) + np.asarray(r[a], float) * np.ones(N) for a in range(m)
]) + src
        J = np.zeros((m * N, m * N))
        for a in range(m):
            for b in range(m):
                J[a * N:(a + 1) * N, b * N:(b + 1) * N] = np.diag(np.asarray(s.Jn[a][b](*u), float) * np
.ones(N)) + (s.D[a] * L if a == b else 0)
        return F, J


def _spline(x, y, robin):
    s0 = -(robin[0] / robin[2]) * (robin[1] - y[0]) if robin else 0.0          # robin = (kap, c, D)
    sl = spline_slopes(x, y[:, None], np.array([s0]), np.zeros(1))[:, 0]
    return cubic_coeffs(x, y[:, None], sl[:, None])[:, :, 0]


def spline_eval(x, y, xq, robin=None, der=False):
    c = _spline(x, y, robin); j = np.clip(np.searchsorted(x, xq) - 1, 0, len(x) - 2); ell = np.diff(x)[j
]; xi = (xq - x[j]) / ell
    if der: return (c[j, 1] + 2 * c[j, 2] * xi + 3 * c[j, 3] * xi ** 2) / ell
    return c[j, 0] + c[j, 1] * xi + c[j, 2] * xi ** 2 + c[j, 3] * xi ** 3


def equidistribute(x, ys, N, Lx=1.0, alpha=0.4, floor=1.0, robins=None):
    xf = np.linspace(0, Lx, 4001); g2 = np.zeros_like(xf)
    for i, y in enumerate(ys): g2 += spline_eval(x, y, xf, (robins or [None] * len(ys))[i], der=True) **
 2
    om = np.convolve(np.sqrt(floor + alpha ** 2 * g2), np.ones(41) / 41, mode="same")
    cum = np.concatenate([[0], np.cumsum(0.5 * (om[1:] + om[:-1]) * np.diff(xf))]); cum /= cum[-1]
    xn = np.interp(np.linspace(0, 1, N), cum, xf); xn[0], xn[-1] = 0.0, Lx; return xn


def steps_geometric(T, h0, hmax, grow=1.02):
    t, hs = 0.0, []
    while t < T - 1e-12:
        h = min(hmax, h0 * grow ** len(hs), T - t); hs.append(h); t += h
    return hs


def rothe(prob, N=101, Lx=1.0, h0=2e-4, hmax=0.02, remesh=10, newton_tol=1e-14):
    m = prob["m"]; rd = RD(prob); A = np.array([[5 / 12, -1 / 12], [3 / 4, 1 / 4]])
    rob = (prob["robin"][0], prob["robin"][1], prob["D"][0]) if prob.get("robin") else None
    robins = [rob] + [None] * (m - 1)
    x = np.linspace(0, Lx, N); y = np.concatenate([f(x) for f in prob["u0"]])
    for _ in range(3):
        x = equidistribute(x, [f(x) for f in prob["u0"]], N, Lx, robins=robins); y = np.concatenate([f(x
) for f in prob["u0"]])
    L, w, src = prob["op"](x); pieces = []; t = 0.0; meshes = [(0.0, x.copy())]
    Lag = np.linalg.inv(np.vander(np.array([0.0, 1 / 3, 1.0]), 3, increasing=True))
    for k, h in enumerate(steps_geometric(prob["T"], h0, hmax)):
        if k > 0 and k % remesh == 0:
            xn = equidistribute(x, [y[a * N:(a + 1) * N] for a in range(m)], N, Lx, robins=robins)
            y = np.concatenate([spline_eval(x, y[a * N:(a + 1) * N], xn, robins[a]) for a in range(m)])
            x = xn; L, w, src = prob["op"](x); meshes.append((t, x.copy()))
        Y = np.concatenate([y, y]); M = m * N
        for _ in range(30):
            F1, J1 = rd.FJ(Y[:M], L, src); F2, J2 = rd.FJ(Y[M:], L, src)
            G = Y - np.concatenate([y, y]) - h * np.concatenate([A[0, 0] * F1 + A[0, 1] * F2, A[1, 0] *
F1 + A[1, 1] * F2])
            d = solve(np.eye(2 * M) - h * np.block([[A[0, 0] * J1, A[0, 1] * J2], [A[1, 0] * J1, A[1, 1]
 * J2]]), G); Y -= d
            if np.max(np.abs(d)) < newton_tol * max(1.0, np.max(np.abs(Y))): break
        cs = (Lag @ np.stack([y, Y[:M], Y[M:]])) / (h ** np.arange(3))[:, None]      # collocation polyn
omial, s-power coefficients
        pieces.append(Piece(t, h, x.copy(), [cs[:, a * N:(a + 1) * N].T for a in range(m)]))
        y = Y[M:].copy(); t += h
    return pieces, meshes


def stdg(prob, N=101, Lx=1.0, q=2, n=1000, newton_tol=1e-14):
    m = prob["m"]; rd = RD(prob); x = np.linspace(0, Lx, N); L, w, src = prob["op"](x)
    y = np.concatenate([f(x) for f in prob["u0"]]); M = m * N
    gx, gw = np.polynomial.legendre.leggauss(q + 3); tg = 0.5 * (gx + 1); wg = 0.5 * gw
    Pg = tg[:, None] ** np.arange(q + 1)[None, :]
    Mt = np.array([[(j / (i + j) if j > 0 else 0.0) for j in range(q + 1)] for i in range(q + 1)])     #
```